

南開大學

區塊鏈實驗六



目录

1 说明	3
1.1 小组成员	3
1.2 仓库	3
2 实验目的	3
3 环境配置	3
4 了解 circom	10
4.1 回答 artifacts/writeup.md 中的相关问题	10
4.1.1 Question 1	10
4.1.2 Question 2	10
4.1.3 Question 3	11
4.2 为 $7 \times 17 \times 19 = 2261$ 创建因子分解证明	11
5 开关电路	12
6 消费电路	13
7 计算花费电路的输入	14
8 赎回证明	16
9 心得体会	17

1 说明

1.1 小组成员

程伟卿	2311865
田园	2313660

表 1: 成员

1.2 仓库

[点击跳转](#)

2 实验目的

在这个实验里，你会学到

- circom, 一个描述算术电路的工具
- snarkjs, 一种用于生成和验证电路满意度的 zk-SNARKs 的工具。

本实验建议使用 Ubuntu 20.04 以上的版本运行。你将使用以下知识来探索私有事务 (private transactions) 的实现：

- 制作一个简单的版本的花费 Tornado 的电路。
- 生成赎回 Tornado 的有效性证明。

3 环境配置

Ubuntu 版本为 24.04.3 LTS, 依次安装 Node.js , npm , snarkjs , circom , mocha 测试工具。最后在 Ex6 文件夹中安装项目本地依赖 (实验打包的依赖库)。

```
1 npm install
2 npm test
```

执行上述指令验证，结果如下：

```

1  mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6$ npm test
2
3  > cs251-cash@0.1.0 test
4  > mocha -s 1s -t 5s test/if_then_else.js test/selective_switch.js
    test/compute_spend_inputs.js test/spend.js
5
6
7
8  IfThenElse
9    1) should give false_value when condition = 0
10   2) should give true_value when condition = 1
11   3) should enforce that s in {0, 1}
12
13  SelectiveSwitch
14   4) should not switch when s = 0
15   5) should switch when s = 1
16   6) should enforce that s in {0, 1}
17
18  computeInput
19   7) transcript0.txt, depth 0, nullifier 1
20   8) transcript1.txt, depth 4, nullifier 4
21   9) transcript2.txt, depth 25, nullifier 7
22
23  Spend
24   witness computable for depth 0
25   witness computable for depth 1
26   witness computable for depth 2
27   10) witness not computable for bad input
28
29
30  3 passing (247ms)
31  10 failing
32
33  1) IfThenElse
34     should give false_value when condition = 0:
35     Error: Signal not assigned: main.out
36     at calculateWitness
      (node_modules/snarkjs/src/calculateWitness.js:63:19)

```

```

37     at Circuit.calculateWitness
      (node_modules/snarkjs/src/circuit.js:59:16)
38   at Context.<anonymous> (test/if_then_else.js:25:30)
39   at process.processImmediate (node:internal/timers:476:21)
40
41 2) IfThenElse
42     should give true_value when condition = 1:
43   Error: Signal not assigned: main.out
44     at calculateWitness
      (node_modules/snarkjs/src/calculateWitness.js:63:19)
45   at Circuit.calculateWitness
      (node_modules/snarkjs/src/circuit.js:59:16)
46   at Context.<anonymous> (test/if_then_else.js:35:30)
47   at process.processImmediate (node:internal/timers:476:21)
48
49 3) IfThenElse
50     should enforce that s in {0, 1}:
51
52   Expected non-binary s to violate constraints
53   + expected - actual
54
55   -Signal not assigned: main.out
56   +Constraint doesn't match
57
58   at Context.<anonymous> (test/if_then_else.js:45:58)
59   at process.processImmediate (node:internal/timers:476:21)
60
61 4) SelectiveSwitch
62     should not switch when s = 0:
63   Error: Signal not assigned: main.out0
64     at calculateWitness
      (node_modules/snarkjs/src/calculateWitness.js:63:19)
65   at Circuit.calculateWitness
      (node_modules/snarkjs/src/circuit.js:59:16)
66   at Context.<anonymous> (test/selective_switch.js:25:30)
67   at process.processImmediate (node:internal/timers:476:21)
68
69 5) SelectiveSwitch
70     should switch when s = 1:

```

```

71     Error: Signal not assigned: main.out0
72     at calculateWitness
       (node_modules/snarkjs/src/calculateWitness.js:63:19)
73     at Circuit.calculateWitness
       (node_modules/snarkjs/src/circuit.js:59:16)
74     at Context.<anonymous> (test/selective_switch.js:36:30)
75     at process.processImmediate (node:internal/timers:476:21)
76
77 6) SelectiveSwitch
78     should enforce that s in {0, 1}:
79
80     Expected non-binary s to violate constraints
81     + expected - actual
82
83     -Signal not assigned: main.out0
84     +Constraint doesn't match
85
86     at Context.<anonymous> (test/selective_switch.js:47:58)
87     at process.processImmediate (node:internal/timers:476:21)
88
89 7) computeInput
90     transcript0.txt, depth 0, nullifier 1:
91
92     AssertionError: expected {} to deeply equal { ... (3) }
93     + expected - actual
94
95     -{}
96     +{
97     +   "digest":
98       "322312280540935210785646262942970941213230064170052589816258546769338952429"
99     +   "nonce": "17"
100    +   "nullifier": "1"
101    +}
102
103     at Context.<anonymous> (test/compute_spend_inputs.js:29:71)
104     at process.processImmediate (node:internal/timers:476:21)
105
106 8) computeInput
    transcript1.txt, depth 4, nullifier 4:

```

```

107
108     AssertionError: expected {} to deeply equal { ...(11) }
109     + expected - actual
110
111     -{}
112     +{
113     +   "digest":
114         "7156595577437608543190201695561556177504424919525785911637562183334393590048"
115     +   "direction[0]": "1"
116     +   "direction[1]": "1"
117     +   "direction[2]": "0"
118     +   "direction[3]": "0"
119     +   "nonce": "14"
120     +   "nullifier": "4"
121     +   "sibling[0]": "3"
122     +   "sibling[1]":
123         "19681602856558162950057707481888122737605614522678761875609416502251241175621"
124     +   "sibling[2]":
125         "4949757766996550399175443158059310394257309534281238094727193345225681506852"
126     +   "sibling[3]":
127         "18633364347856320646884886547836821398229701470730139821384419637975014912921"
128     + }
129
130     at Context.<anonymous> (test/compute_spend_inputs.js:29:71)
131     at process.processImmediate (node:internal/timers:476:21)
132
133     9) computeInput
134         transcript2.txt, depth 25, nullifier 7:
135
136         AssertionError: expected {} to deeply equal { ...(53) }
137         + expected - actual
138
139         -{}
140         +{
141         +   "digest":
142             "12008438219023705767548476986097257442459718653812944591700422139092737647616"
143         +   "direction[0]": "0"
144         +   "direction[10]": "0"
145         +   "direction[11]": "0"

```

```

141 + "direction[12]": "0"
142 + "direction[13]": "0"
143 + "direction[14]": "0"
144 + "direction[15]": "0"
145 + "direction[16]": "0"
146 + "direction[17]": "0"
147 + "direction[18]": "0"
148 + "direction[19]": "0"
149 + "direction[1]": "1"
150 + "direction[20]": "0"
151 + "direction[21]": "0"
152 + "direction[22]": "0"
153 + "direction[23]": "0"
154 + "direction[24]": "0"
155 + "direction[2]": "1"
156 + "direction[3]": "0"
157 + "direction[4]": "0"
158 + "direction[5]": "0"
159 + "direction[6]": "0"
160 + "direction[7]": "0"
161 + "direction[8]": "0"
162 + "direction[9]": "0"
163 + "nonce": "17"
164 + "nullifier": "7"
165 + "sibling[0]": "0"
166 + "sibling[10]":
    "18282053668740841793538385484375298138297506270747903022149301437302461790269"
167 + "sibling[11]":
    "11854931242116648486981320255420813279877710904852643395113277376779758855098"
168 + "sibling[12]":
    "9381451415888087631316092497966048521515156819026912291039253890498276841218"
169 + "sibling[13]":
    "12100772402100122546174508383775022952488245732542301225643449270292781349246"
170 + "sibling[14]":
    "11678712791072686349426619739236716758901691981625860855979566582851944885050"
171 + "sibling[15]":
    "3770348298675346367043162266532824235411923189155139302762981231467086905777"
172 + "sibling[16]":
    "15393217232339109119447654107682029415590614055737384965585789826144870052964"

```



```

173 + "sibling[17]":
      "12264910519433830970373596092952995265084876820351049594390604200874182660748"
174 + "sibling[18]":
      "13210202697341541829004339447849238228437591615691467850843528694875750643668"
175 + "sibling[19]":
      "12275175546571410475065038446548306212751740664852829804344756202168280893476"
176 + "sibling[1]":
      "12189038830210795421670700154279045415080606790787559391259916858881301644792"
177 + "sibling[20]":
      "11260233943172690917153305792393889153843582237797805965260892635012622603738"
178 + "sibling[21]":
      "2961348222531884875876351277329553867656725922678488160549612425483901069836"
179 + "sibling[22]":
      "19422277424773577396487364828210371594978232971749498960597640771480769104504"
180 + "sibling[23]":
      "21441775425524591870038545453134036680101698299599460058222532069207657753639"
181 + "sibling[24]":
      "11649977709137748592205755176624485867846310258853416849643547130466466234844"
182 + "sibling[2]":
      "458006536376401925134706847632908249386803523308752072267916752010455047876"
183 + "sibling[3]":
      "18633364347856320646884886547836821398229701470730139821384419637975014912921"
184 + "sibling[4]":
      "1428163557486472957671755266439110187892884941794525091690272793864921825097"
185 + "sibling[5]":
      "19187735370145708415164400941549267147777572879769686202539715761761805515060"
186 + "sibling[6]":
      "1354502918524251823186298550250875206163022924074772172859405589330217709272"
187 + "sibling[7]":
      "5177479716332055346633334667550188310219858116800539706838435929857232894550"
188 + "sibling[8]":
      "2339305980315949773396963743270218668182561894986101180771293554269664783865"
189 + "sibling[9]":
      "10931860064776657815956113554498305834898340788173502696934655289858000589974"
190 +}
191
192 at Context.<anonymous> (test/compute_spend_inputs.js:29:71)
193 at process.processImmediate (node:internal/timers:476:21)
194

```

```

195 10) Spend
196     witness not computable for bad input:
197     AssertionError: Expected bad inputs to crash witness computation:
198         expected [Function] to throw Error
        at Context.<anonymous> (test/spend.js:33:58)

```

测试结果符合预期：3 个测试通过，10 个测试失败，说明环境配置已完全成功（失败的测试均依赖未实现的电路和函数，属于正常情况）！

4 了解 circom

4.1 回答 artifacts/writeup.md 中的相关问题

在 writeup.md 中回答，这里再展示一遍结果：

4.1.1 Question 1

在 Circom 电路中， $(2 ** i)$ 并不是一个变量或信号，而是一个**编译时常量**（编译时已知的确定数值）。因此，表达式 $(2 ** i) * \text{bits}[i]$ 实际上是：

- $\text{bits}[i]$ ：一个信号变量
- $(2 ** i)$ ：一个已知的常数系数

在这里对于每个索引 i ， $(2 ** i)$ 是在电路编译时确定的固定值， $\text{bits}[i]$ 是电路中实际的信号变量，整个表达式 `sum_of_bits` 是形式为： $c_1 \cdot s_1 + c_2 \cdot s_2 + \dots + c_n \cdot s_n$ 的线性组合，其中 c_i 是常数， s_i 是信号。

R1CS (Rank-1 Constraint System) 要求约束是线性的，但允许信号与常数的乘法。因为常数在约束系统中不会引入新的变量维度，所以这样的表达式仍然构成秩为 1 的约束。

4.1.2 Question 2

在 Circom 中，`<==` 运算符是一个**复合运算符**，它同时执行两个操作：

1. **信号赋值**：将右侧表达式的值赋给左侧信号

2. **约束生成**: 自动创建一个等式约束, 强制左侧信号必须等于右侧表达式

具体来说, $a \leq b + c$ 等价于:

```
1 a === b + c; // 创建约束: a 必须等于 b + c
2 a <-- b + c; // 赋值: 将 b+c 的值赋给 a
```

简而言之, $\leq$ 用于**中间信号** (非输入/输出信号), 它确保电路执行时的实际计算值 (赋值) 与电路的约束系统 (证明验证) 保持一致。这是 Circom 中连接”计算”和”证明”的关键机制, 保证 prover 不能随意赋值而必须满足约束。

4.1.3 Question 3

这个代码片段无效, 原因如下:

a、b 和 c 都是电路信号, 第四行代码对信号 a 做了位运算。circom 中信号的赋值与约束操作是分开进行的, 不能直接在赋值语句中使用位运算。

所以应该对其进行修改: 先执行 $a \leq (a \& 1) * b$ 语句来完成位运算并将结果乘上 b。接着再使用约束语句 $c === a$ 来完成赋值操作。

4.2 为 $7 \times 17 \times 19 = 2261$ 创建因子分解证明

第一步, 编译电路为 JSON 格式, 执行下面指令:

```
1 circom ./circuits/example.circom -o ./artifacts/circuit.json
```

第二步, 复制电路文件到当前主目录。

第三步, 生成 SNARK 密钥对。执行以下命令初始化 setup, snarkjs 会自动读取当前目录的 circuit.json, 生成 proving_key.json (证明密钥) 和 verification_key.json (验证密钥):

```
1 snarkjs setup
```

第四步, 创建输入文件 input.json :

```
1 {
2   "product": 2261,
3   "factors": [7, 17, 19]
```

```
4 }
```

第五步，执行以下命令生成见证文件：

```
1 snarkjs calculatewitness
```

第六步，执行以下命令生成因子分解证明：

```
1 snarkjs proof
```

第七步，执行命令 `snarkjs verify` 验证证明，终端输出 OK（表示证明有效）：

```
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6$ snarkjs verify
OK
```

5 开关电路

```
1 template IfThenElse() {
2     signal input condition;
3     signal input true_value;
4     signal input false_value;
5     signal output out;
6
7     // 辅助信号：构建线性约束，符合hint要求
8     signal helper;
9     // 约束condition只能是0或1
10    condition * (1 - condition) == 0;
11    // 线性组合实现条件选择
12    helper <== condition * (true_value - false_value);
13    out <== false_value + helper;
14 }
```

```
1 template SelectiveSwitch() {
2     signal input in0;
3     signal input in1;
4     signal input s;
5     signal output out0;
```

```

6     signal output out1;
7
8     // 约束s只能是0或1
9     s * (1 - s) === 0;
10    // 线性约束实现开关逻辑
11    signal diff;
12    diff <== in1 - in0;
13    out0 <== in0 + s * diff;
14    out1 <== in1 - s * diff;
15 }

```

上述代码展示了在 Circom 中实现开关逻辑。IfThenElse 实现了三路选择器功能，通过约束 condition 为 0 或 1，然后使用线性组合 $out = false_value + condition * (true_value - false_value)$ 实现条件选择。SelectiveSwitch 实现了一个双向开关，当 $s=0$ 时输出保持不变 ($out0=in0$, $out1=in1$)，当 $s=1$ 时交换两个输出 ($out0=in1$, $out1=in0$)。两个电路都完全使用线性约束实现，符合 R1CS 约束系统的要求。

6 消费电路

```

1 template Spend(depth) {
2     signal input digest;
3     signal input nullifier;
4     signal private input nonce;
5     signal private input sibling[depth];
6     signal private input direction[depth];
7
8     component computed_hash[depth + 1];
9     computed_hash[0] = Mimc2();
10    computed_hash[0].in0 <== nullifier;
11    computed_hash[0].in1 <== nonce;
12
13    component switches[depth];
14    for(var i = 0; i < depth; ++i){
15        switches[i] = SelectiveSwitch();
16        switches[i].in0 <== computed_hash[i].out;
17        switches[i].in1 <== sibling[i];

```

```

18         switches[i].s <== direction[i];
19
20         computed_hash[i+1] = Mimc2();
21         computed_hash[i+1].in0 <== switches[i].out0;
22         computed_hash[i+1].in1 <== switches[i].out1;
23     }
24     computed_hash[depth].out === digest;
25 }

```

上述代码实现了一个 Merkle 树验证和路径重构的逻辑。电路从 nullifier 和 nonce 的哈希值开始,通过 depth 层的 SelectiveSwitch 开关组件和 Mimc2 哈希组件,沿着给定的 sibling 节点和 direction 方向逐步向上计算 Merkle 树路径。每个 SelectiveSwitch 根据 direction[i] 的值 (0 或 1) 决定当前哈希值与 sibling 节点的顺序,确保正确的 Merkle 路径构建。最终计算的根哈希值必须与输入的 digest 相等,从而验证了 nullifier 和 nonce 确实存在于指定的 Merkle 树中,且用户知道正确的路径信息。

7 计算花费电路的输入

设计 computeInput 函数,代码如下:

```

1 function computeInput(depth, transcript, nullifier) {
2     // 1. 初始化稀疏Merkle树和核心变量
3     const merkleTree = new SparseMerkleTree(depth);
4     let targetNonce = null;
5     let targetCommitment = null;
6
7     // 2. 遍历交易转录列表,处理每个硬币并插入Merkle树
8     for (const coinItem of transcript) {
9         let currentCommitment;
10
11         //
12         处理两种硬币格式: 单元素 (直接是硬币)、双元素 (nullifier+nonce, 需计算哈希)
13         if (coinItem.length === 1) {
14             currentCommitment = BigInt(coinItem[0]).toString(); //
15             转为大数字符串,避免精度丢失
16         } else if (coinItem.length === 2) {

```

```

15     const [currNullif, currNonce] = coinItem;
16     // 计算MIMC2哈希作为硬币承诺
17     currentCommitment = mimc2(BigInt(currNullif),
        BigInt(currNonce)).toString();
18
19     // 匹配目标nullifier, 记录对应的nonce和承诺
20     if (currNullif === nullifier) {
21         targetNonce = currNonce;
22         targetCommitment = currentCommitment;
23     }
24     } else {
25         throw new Error(无效的硬币格式: $coinItem.join(" "), 仅支持 1 个
            或 2 个元素);
26     }
27
28     // 将当前承诺插入Merkle树
29     merkleTree.insert(BigInt(currentCommitment));
30 }
31
32 // 3. 校验目标nullifier是否找到
33 if (!targetCommitment || !targetNonce) {
34     throw new Error(在转录文件中未找到目标 nullifier: $nullifier);
35 }
36
37 // 4. 获取目标承诺在Merkle树中的路径 (兄弟节点+方向)
38 const merklePath = merkleTree.path(BigInt(targetCommitment));
39
40 // 5. 构造电路输入结果对象
41 const spendInput = {
42     "digest": merkleTree.digest.toString(),
43     "nullifier": nullifier,
44     "nonce": targetNonce
45 };
46
47 // 6. 补充sibling和direction数组 (适配电路输入格式)
48 for (let i = 0; i < depth; i++) {
49     const [siblingNode, isLeftSibling] = merklePath[i];
50     spendInput[sibling[$i]] = siblingNode.toString();
51     spendInput[direction[$i]] = isLeftSibling ? "1" : "0"; //

```

```

        布尔值转字符串数字
52     }
53
54     return spendInput;
55 }

```

执行命令 `npm test` 运行所有测试，验证 `computeInput` 相关测试是否通过。

13 个测试用例全部通过，`computeInput` 的 3 个测试不再失败，返回预期的输入数据对象：

```

mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6$ npm test

> cs251-cash@0.1.0 test
> mocha -s 1s -t 5s test/if_then_else.js test/selective_switch.js test/compute_spend_inputs.js test/spend.js

IfThenElse
  ✓ should give `false_value` when `condition` = 0
  ✓ should give `true_value` when `condition` = 1
  ✓ should enforce that s in {0, 1}

SelectiveSwitch
  ✓ should not switch when s = 0
  ✓ should switch when s = 1
  ✓ should enforce that s in {0, 1}

computeInput
  ✓ transcript0.txt, depth 0, nullifier 1
  ✓ transcript1.txt, depth 4, nullifier 4
  ✓ transcript2.txt, depth 25, nullifier 7

Spend
  ✓ witness computable for depth 0
  ✓ witness computable for depth 1
  ✓ witness computable for depth 2 (818ms)

13 passing (2s)

```

8 赎回证明

第一步，生成电路输入文件：

```

1 node src/compute_spend_inputs.js 10
  './test/compute_spend_inputs/transcript3.txt' 10137284576094 -o
  input.json

```

第二步，将 `input.json` 复制到电路目录，确保后续编译和见证计算能读取。

第三步，编译深度 10 的 Spend 电路（生成 circuit.json）：

```
1 cd ../test/circuits/ # 切换到电路目录
2 circom spend10.circom -o circuit.json # 编译电路，输出circuit.json
```

第四步，生成 SNARK 密钥对（proving_key + verification_key）：

```
1 snarkjs setup
```

第五步，基于 circuit.json 和 input.json 计算见证：

```
1 snarkjs calculatewitness
```

第六步，生成赎回证明（proof.json）：

```
1 snarkjs proof
```

第七步，验证证明：

```
1 snarkjs verify
```

```
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6/test/circuits$ circom spend10.circom -o circuit.json
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6/test/circuits$ snarkjs setup
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6/test/circuits$ snarkjs calculatewitness
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6/test/circuits$ snarkjs proof
mojo@LAPTOP-B8ONCE05:/mnt/c/Users/mojo/Desktop/Ex6/test/circuits$ snarkjs verify
OK
```

9 心得体会

通过本次实验，学会零知识证明如何将复杂的交易逻辑转化为严密的算术电路约束，在保护隐私的同时确保可验证性。从设计基础选择电路到构建完整的 Merkle 树验证系统，每一步都认识到密码学工程中数学严谨性与实践创新的精妙结合，同时也引发对隐私技术社会影响的思考。